

The Service Desk Command Center

From one stalled ticket to a governed, self-driving service operation.

Outcome: MTTR, SLA compliance, auto-resolution rate, CSAT

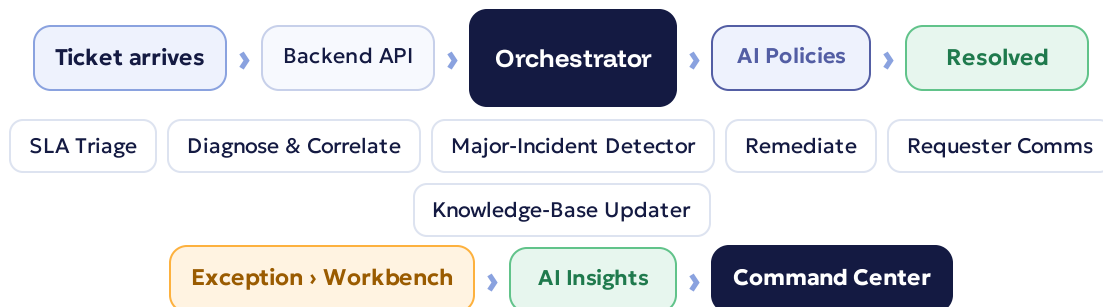
Round 1 track: The Stalled-Ticket Resolver

What changed since Round 1. Your Round 1 AI Employee is still the brain and is not thrown away. Round 2 puts a real operation around it: the agent grows to five or more Operators on Auto, and you build a coded Command Center on the template with AI Policies, AI Insights, an AI Manager, a Data Manager, and a Workbench. The bar moves from "can you decompose the problem" to "can you run a governed operation."

THE SCENARIO

Arjun no longer nudges one ticket forward; he runs an IT service desk taking work from email, chat, and a portal, with SLAs that respect business hours and holidays, and teams that depend on each other to close anything. Thirty tickets land in twenty minutes and all trace to one failing system. A VIP request stalls after hours, minutes from breach. A quick fix needs a change approval before anyone may touch production. A remediation fails and must be rolled back cleanly. The Round 2 build runs the whole queue, spots the major incident hiding inside a flood of tickets, and hands a person a clean queue for the risky calls.

HOW THE AI EMPLOYEE WORKS (EXAMPLE)



The Orchestrator prioritizes by SLA risk, branches on whether a fix is safe to automate, and holds state to link related tickets into one incident.

This is one example, not a required design. Add or remove Operator Agents, change how each one works and the logic behind it, make the flow more optimal, and make sure it handles every kind of exception. Teams differ on the how; judges compare on the result.

WHAT RUNS WHERE

LAYER	RUNS ON	WHAT IT IS
The agent	Auto	One Orchestrator plus five or more Operator Agents doing the actual service desk work. Orchestration must be built on Supervity Auto.
Command Center	Template	The live operational picture: KPIs, agent status, exception queues.
AI Policies	Template	Configurable business rules that constrain what the agent may do alone.
AI Insights	Template	Patterns, anomalies, and recommendations from the operational data.
AI Manager	Template	Chat and orchestration surface that talks to the Auto agent.
Data Manager	Template	Live registry of every connected system and its health.
Workbench	Template	The human queue where exceptions are reviewed and cleared.

Start from the template: github.com/digitamizers/AutoPilot-Template. Clone it and run `docker compose up` for the backend, database, and Command Center shell. Generate your Workflow API key at auto.supervity.ai/u/api-keys and read auto.supervity.ai/docs before wiring. Full walkthrough in the Round 2 Participant Guide.

WHAT YOU BUILD

The agent on Auto

One Orchestrator plus five or more Operators doing the real work. Extend your Round 1 build.

Command Center

Live view of backlog by SLA risk, mean time to resolution, SLA compliance, auto-resolution rate, open major incidents, and CSAT trend.

AI Policies engine

At least three editable rules: which fixes are safe to auto-remediate, SLA thresholds by priority and business hours, VIP prioritization, change-approval requirements, and the escalation matrix.

AI Insights

Recurring known-error clusters across users, a major incident forming from many tickets, knowledge gaps where no article exists, SLA-breach forecasts, and uneven team load.

AI Manager

A chat and orchestration surface where a person asks the operation questions and watches the Orchestrator delegate to Operators and report back.

Data Manager

A live registry of every connected system, what each is used for, and whether the connection is healthy.

The Workbench

The human queue for risky remediation, change approvals, and major-incident command, each with the diagnosis, the correlated tickets, and the agent's recommendation.

WHAT YOU CAN ADD IN ROUND 2

Round 1 covered the core path. These are the business functions this operation is still missing, and each one is naturally its own Operator. Pick the ones that make your build a fuller service desk AI Employee; you are not required to build all of them.

Major-incident command

Detect a flood of related tickets, open a parent incident, and run the comms until it closes.

Change approval

Request and track the approval a fix needs before anything touches production.

Rollback and verification

Verify a remediation actually worked, and roll it back cleanly when it does not.

Knowledge-base authoring

Write an article from a resolved ticket so the next identical request deflects itself.

CSAT follow-up

Survey the requester after resolution and escalate a poor score to a human.

Load balancing and on-call routing

Assign work by team capacity, shift, and the on-call roster rather than round robin.

WORKING WITH THE DATA

The dataset is how you exercise your integrations, not a file to read from. Either seed the systems you connect with it (load the records into your CRM, ticket tool, database, or an Airtable base), or park it in a store like SharePoint, OneDrive, Box, or Dropbox and point an integration at it, then pull the data back through that integration. Either way the data reaches your Operators through a live integration, at least three: one channel, one system of record, and a live exception to a person.

A way in

A new or updated ticket via email or a channel.

A system of record

The ticket backlog in Jira; an access register in Airtable or Supabase.

A human loop

A Slack escalation before any risky remediation, routed through the Workbench.

Note: Google integrations (Drive, Sheets, Gmail, Calendar) are in beta for this event, so use alternatives such as Airtable, Supabase, SharePoint, OneDrive, Box, Dropbox, and Outlook.

THE ROUND 2 DATA PACK

WHAT IS IN IT

Ten tables and roughly 1,200 rows of Jira Service Management-style data: the issue backlog with SLA fields, a user directory, an access register, and a knowledge base, plus the Round 2 additions: Ticket_Comments, CSAT_Surveys, Change_Requests, Incident_Problem_Links, SLA_Calendar, and Team_Roster.

WHAT IS NEW SINCE ROUND 1

Six new tables: Ticket_Comments, CSAT_Surveys, Change_Requests, Incident_Problem_Links, SLA_Calendar, and Team_Roster, covering conversation threads, satisfaction, change control, incident linkage, business-hours SLA, and team load. The pack keeps the Round 1 schema, so your existing build still reads it, and adds volume and harder cases. Build for the general case: a judge may ask you to run a record you did not prepare.

MANDATORY IN ROUND 2

- 1. Agent on Auto.** One Orchestrator coordinating at least five distinct Operators, with parallel, branching, or stateful behavior.
- 2. Connected Command Center.** The coded frontend is wired to the Auto agent through the backend API and shows live agent activity, not the template's demo data.
- 3. Working AI Policies.** At least three active policies that measurably change agent behavior, editable without code, with every evaluation logged.
- 4. A live human loop.** At least one real exception routed to the Workbench with full context and resolved there.
- 5. Live integrations.** At least three across two categories (one channel, one system of record), visible and healthy in the Data Manager.

DO'S AND DON'TS

Do

- Build the Operators on Auto; code the Command Center, Policies, Insights, Manager, and Data Manager around them.
- Make policies genuinely dynamic: a changed threshold changes what the agent does, live.
- Generate insights from the data the agent actually processed.
- Route real exceptions to the Workbench with full context and a recommendation.

Don't

- Rebuild the Orchestrator or Operators outside Supervity Auto.
- Ship policy or insight pages that only show the template's static demo data.
- Let the agent invent a value on a missing field; pause and escalate.
- Hardcode to these rows, skip the human loop, or stage the demo.

Bonus. Going beyond the brief earns bonus points once the stated problem is solved: extra Operators and richer downstream actions, forecasting, self-learning where a human correction at the Workbench changes future behavior, deeper auditability, meaningful use of open-source components given a real role in the build, and genuine creativity on top of the problem statement.

Not apps. Not agents. AI Employees run the Autos.